

Functional requirement

Capture — pull raw packets off a chosen interface (WiFi monitor mode, Ethernet, or BLE advertisement scanning)

Parse/classify — extract metadata: src/dst, protocol, size, timestamp, device identifiers

Store — persist captures (pcap file or a lightweight DB) for later query

Filter/query — by protocol, IP/MAC, time window, device

Present — CLI or minimal web dashboard: live packet rate, protocol breakdown, top talkers

Detect (stretch) — flag suspicious patterns (ARP spoofing, rogue AP, plaintext creds, unknown device joining)

Report/export — summary report (CSV/PDF) of a capture session

Non-functional requirements

Modularity — capture, parsing, storage, and presentation should be separable layers (this is where SE grading usually lives)

Extensibility — adding a new protocol analyzer or a new capture backend (say, adding BLE later) shouldn't touch unrelated code

Portability — runs on constrained hardware (Pi/ESP), so mind memory/CPU budget

Reliability — no packet-loss under moderate load; graceful handling of capture-permission or interface failures

Maintainability - Code must be organized by layer with documented interfaces, include unit tests for parsers and core logic, and follow a consistent style so a new contributor can extend one layer without reading the whole codebase

Security- Captured traffic may contain sensitive data (credentials, personal traffic); stored captures must be access-controlled on disk, and the tool must only capture traffic the user is authorized to monitor (own network/devices).

Usability--The CLI/dashboard output must be readable without prior tool-specific training — clear labels, sane defaults, no manual config file editing for basic use.